

Advanced MCP Applications with Streamable HTTP, Roots, and Sampling

Estimated time needed: 60 minutes

In this lab, you'll build an advanced MCP application using HTTP transport for remote connectivity, filesystem roots for security boundaries, and sampling for server-initiated LLM requests. You'll create a **base HTTP client** with HTTP protocol support, then build two applications on top of it: a **GUI client app** for manual interaction and an **AI-powered host app** with full remote MCP capabilities.

This design demonstrates production-ready patterns for remote MCP servers with enterprise-grade security.

Learning Objectives

After completing this lab, you will be able to:

- Connect to remote MCP servers using HTTP transport
- Configure filesystem roots for secure file access boundaries
- Implement sampling approval workflows for server-initiated LLM requests
- Build applications that inherit from an HTTP-capable base client
- Understand when to use HTTP vs STDIO transport
- Handle security boundaries and user approval flows

Prerequisites

Before starting this lab, you should have:

- Experience building MCP clients with base/derived class architecture
- Basic Python programming knowledge

Understanding of HTTP transport

- Familiarity with async/await patterns in Python
- Basic knowledge of object-oriented programming (inheritance)
- Awareness of filesystem security concepts

Lab Setup

Let's set up your development environment for advanced MCP features.

Create Virtual Environment

 bash 

```
python3.11 -m venv mcp_advanced_env  
source mcp_advanced_env/bin/activate
```

▶ Run

Install Dependencies

Install the MCP SDK, FastMCP, HTTP client libraries, Gradio, and OpenAI:

 bash 

```
pip install mcp==1.16.0 fastmcp==2.12.5 httpx==0.28.1 uvicorn==0.38.0 grad
```

▶ Run

Create Project Structure

 bash 

```
mkdir mcp_advanced_lab  
cd mcp_advanced_lab  
mkdir workspace
```

▶ Run

The **workspace** directory will serve as our roots-protected filesystem boundary.

Understanding Key Concepts

Before we begin, let's understand the three key concepts in this lab.

HTTP Transport

What it is:

- Remote server runs independently (not as subprocess)
- Communication via bidirectional HTTP transport
- Server can handle multiple clients simultaneously
- Stateful or stateless connections

When to use:

- Production deployments
- Remote/cloud services
- Microservices architectures
- Shared server resources
- Multi-client scenarios

vs STDIO Transport:

- STDIO: Local subprocess, low latency, simple debugging
- HTTP: Remote process, network latency, production-ready

Filesystem Roots

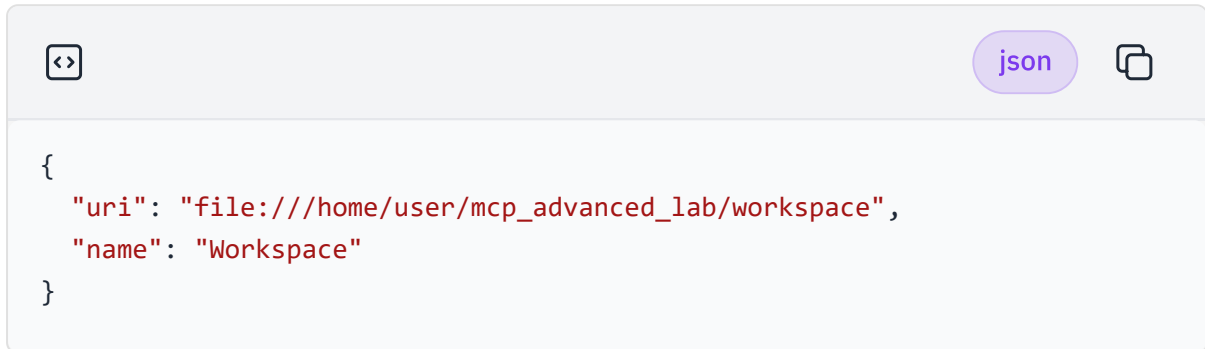
What they are:

- A **client capability** that exposes allowed directories to servers
- Client declares which filesystem locations are accessible
- Server requests the list via `roots/list` JSON-RPC method

MCP Protocol Flow:

1. Client initializes and declares `roots` capability
2. Server sends `roots/list` request to client
3. Client responds with list of allowed `file://` URIs
4. Server checks all file operations against these URIs
5. Client can notify server when roots change (optional)

Example Root Definition:



```
{
  "uri": "file:///home/user/mcp_advanced_lab/workspace",
  "name": "Workspace"
}
```

Security benefit:

- Client maintains full control over filesystem access
- Server cannot access files outside exposed roots
- Essential for enterprise deployments with sensitive data
- Prevents path traversal and unauthorized access

Sampling (Server-Initiated LLM Requests)

What it is:

- MCP protocol feature where servers request LLM completions from clients
- Uses JSON-RPC method: `sampling/createMessage`
- Client maintains control over model selection, API keys, and costs
- Enables agentic behavior without servers needing LLM access

MCP Protocol Flow:

1. Client initializes and declares `sampling` capability
2. Server sends `sampling/createMessage` JSON-RPC request with:

- Messages array (conversation history)
 - Model preferences (optional)
 - System prompt (optional)
 - Max tokens (required)
3. Client shows approval dialog to user (human-in-the-loop)
 4. If approved, client calls LLM with provided parameters
 5. Client sends JSON-RPC response with LLM result
 6. Server uses result to complete its operation

Benefits:

- Server doesn't need LLM API keys
- Client controls model selection and costs
- Privacy maintained at client side
- Standardized protocol for LLM requests

Security consideration:

- MUST have human-in-the-loop approval
- User sees server's prompt before LLM execution
- Prevents unauthorized LLM usage
- Client can reject any sampling request

Build the HTTP MCP Server

Now let's create an HTTP-based MCP server with roots-aware file operations and sampling capabilities.

Click below to open the file:

Open mcp_http_server.py in IDE

Step 1: Add imports and configuration

```
<> python 
```

```
from fastmcp import FastMCP
from pathlib import Path
import logging
import warnings

# Suppress deprecation warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
logging.getLogger("fastmcp").setLevel(logging.WARNING)

mcp = FastMCP("HTTP File Server")
```

What this does: Imports FastMCP for the server, Path for file operations, and suppresses verbose logging and deprecation warnings for cleaner output.

Step 2: Configure roots and base directory

```
<> python 
```

```
BASE_DIR = Path(__file__).parent / "workspace"
BASE_DIR.mkdir(exist_ok=True)
```

What this does: Defines the base directory as `workspace/` relative to the server script. This directory represents our roots boundary - the server will only operate within this directory. Creates the directory if it doesn't exist.

Security note: All file operations will be validated against this base directory to prevent path traversal attacks.

Step 3: Add roots-aware helper function

```
def is_within_roots(path: Path) -> bool:
    """Check if path is within allowed roots directory."""
    try:
        path.resolve().relative_to(BASE_DIR.resolve())
        return True
    except ValueError:
        return False
```

What this does: Security function that checks if a given path is within the allowed roots directory. Uses `resolve()` to handle symlinks and `relative_to()` to verify the path is a subdirectory of `BASE_DIR`. Returns `False` if path escapes roots boundary.

Why this matters: Prevents path traversal attacks such as `../../../../../etc/passwd`. Essential for production security.

Step 4: Add read_file tool

```
@mcp.tool()
def read_file(filepath: str) -> str:
    """Read a file from the workspace directory."""
    path = BASE_DIR / filepath

    if not is_within_roots(path):
        return f"Error: Access denied - path outside workspace roots"

    if not path.exists():
        return f"Error: File not found: {filepath}"

    try:
        content = path.read_text()
        return content
    except Exception as e:
        return f"Error reading file: {str(e)}"
```


What this does: Tool for reading files with roots checking:

1. Constructs full path relative to BASE_DIR
2. Checks if path is within roots - returns error if not
3. Verifies file exists
4. Reads and returns file content
5. Error handling for read failures

Security: The `is_within_roots()` check ensures clients cannot read files outside the workspace directory.

Step 5: Add write_file tool

A code editor window with a light gray header bar. On the left is a code icon, on the right is a 'python' language selector and a copy icon. The main area contains Python code for an MCP tool.

```
@mcp.tool()
def write_file(filepath: str, content: str) -> str:
    """Write content to a file in the workspace directory."""
    path = BASE_DIR / filepath

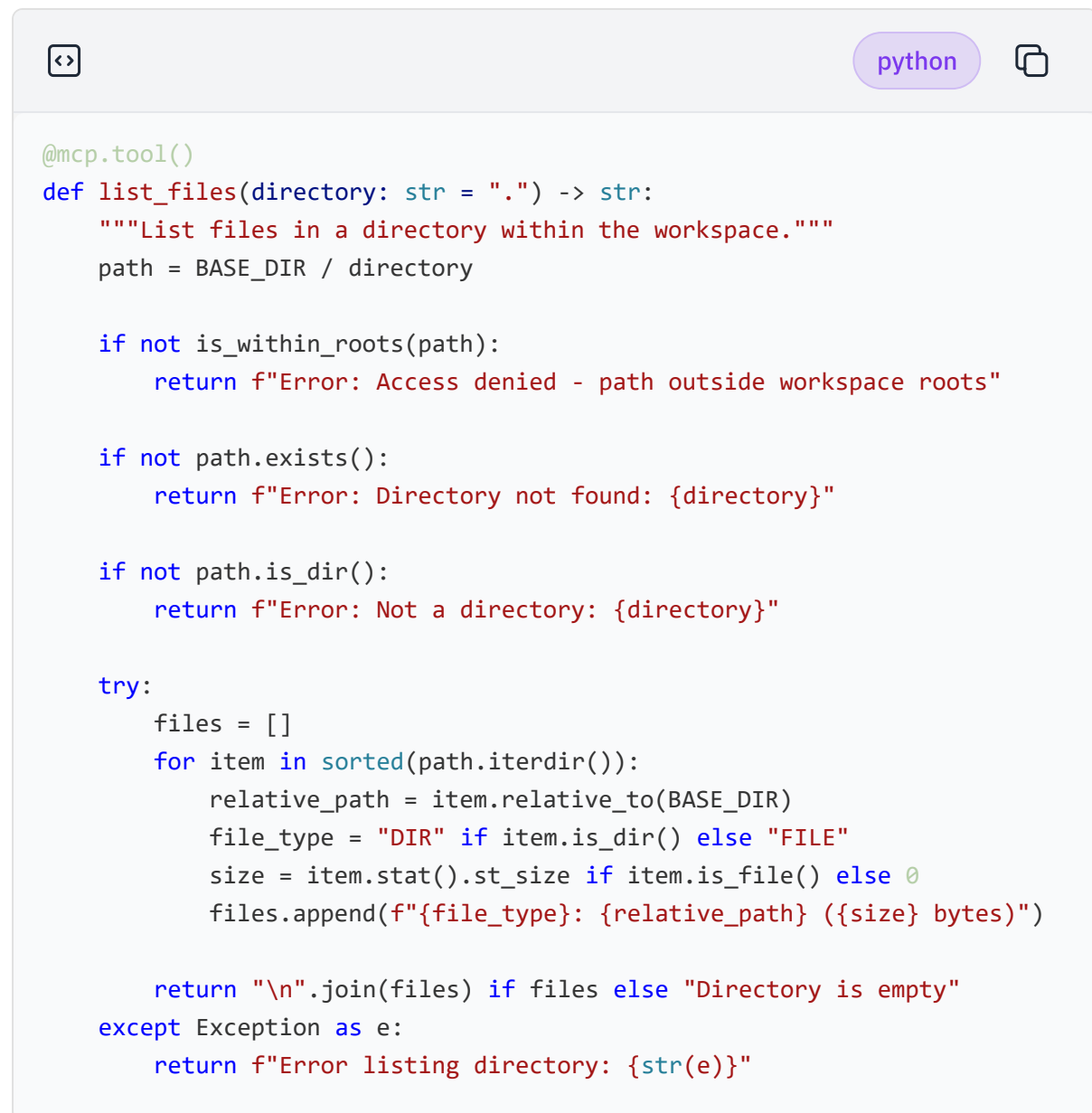
    if not is_within_roots(path):
        return f"Error: Access denied - path outside workspace roots"

    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(content)
        return f"Successfully wrote {len(content)} characters to {filepath}"
    except Exception as e:
        return f"Error writing file: {str(e)}"
```

What this does: Tool for writing files with roots checking:

1. Constructs full path relative to BASE_DIR
2. Checks if path is within roots
3. Creates parent directories if needed
4. Writes content to file
5. Returns success message with character count
6. Error handling for write failures

Step 6: Add list_files tool



```
@mcp.tool()
def list_files(directory: str = ".") -> str:
    """List files in a directory within the workspace."""
    path = BASE_DIR / directory

    if not is_within_roots(path):
        return f"Error: Access denied - path outside workspace roots"

    if not path.exists():
        return f"Error: Directory not found: {directory}"

    if not path.is_dir():
        return f"Error: Not a directory: {directory}"

    try:
        files = []
        for item in sorted(path.iterdir()):
            relative_path = item.relative_to(BASE_DIR)
            file_type = "DIR" if item.is_dir() else "FILE"
            size = item.stat().st_size if item.is_file() else 0
            files.append(f"{file_type}: {relative_path} ({size} bytes)")

        return "\n".join(files) if files else "Directory is empty"
    except Exception as e:
        return f"Error listing directory: {str(e)}"
```

What this does: Tool for listing directory contents with roots checking:

1. Constructs full path relative to BASE_DIR
2. Checks if path is within roots
3. Verifies path exists and is a directory
4. Iterates through directory items
5. Formats output showing type (FILE/DIR), relative path, and size
6. Returns sorted list or "empty" message
7. Error handling for listing failures

Details: Shows relative paths (from BASE_DIR) rather than absolute paths for cleaner output.

Step 7: Add analyze_code tool

python

```
@mcp.tool()
def analyze_code(code: str, focus: str = "quality") -> str:
    """Analyze code focusing on specified aspect.

    In a full MCP implementation with bidirectional communication,
    this tool would send a sampling/createMessage JSON-RPC request
    to the client. For this educational lab, we return a message
    indicating where sampling would occur.
    """
    return f"[SAMPLING TRIGGER]
    This tool would send a sampling/createMessage request to the client:

    {{
      'method': 'sampling/createMessage',
      'params': {{
        'messages': [{{'role': 'user', 'content': {{
          'type': 'text',
          'text': 'Analyze this code for {focus}:\\n{code[:50]}...'
        }}}}],
        'maxTokens': 500
      }}
    }}

    The client would:
    1. Show approval dialog to user
    2. If approved, call LLM with the prompt
    3. Return LLM response to server
    4. Server would use response to complete analysis

    Note: Full bidirectional sampling requires low-level MCP SDK.
    This simplified version demonstrates the concept."""
```

What this does: Demonstrates where MCP sampling would be used:

1. Takes code and focus area as parameters
2. Shows the MCP `sampling/createMessage` request that would be sent
3. Explains the sampling workflow
4. Returns educational message about the protocol

MCP Sampling Protocol: In production MCP implementations, this tool would:

1. Send JSON-RPC `sampling/createMessage` request to client via session

2. Include messages array with prompt
3. Specify maxTokens and optional model preferences
4. Wait for client's JSON-RPC response with LLM result
5. Use LLM response to complete the analysis

Why simplified: Full bidirectional JSON-RPC sampling requires session management not easily accessible in FastMCP's tool decorators. This demonstrates the concept for educational purposes.

Step 8: Add resource template



```
@mcp.resource("file://workspace/{filename}")
def get_workspace_file(filename: str) -> str:
    """Read a file from the workspace as a resource."""
    path = BASE_DIR / filename

    if not is_within_roots(path):
        raise ValueError(f"Access denied - path outside workspace roots")

    if not path.exists():
        raise ValueError(f"File not found: {filename}")

    return path.read_text()
```

What this does: Resource template for accessing workspace files:

1. Uses URI pattern `file://workspace/{filename}`
2. Checks roots boundary
3. Verifies file exists
4. Returns file content
5. Raises ValueError for errors (MCP resource error handling)

Resources vs Tools: Resources are for passive data access (read-only), tools are for active operations (read/write/execute).

Step 9: Add prompt templates

```
 python   
  
@mcp.prompt()  
def review_code(filename: str) -> str:  
    """Generate a prompt to review code from a file."""  
    return f"""Please review the code in file '{filename}' and provide:  
  
1. A summary of what the code does  
2. Potential bugs or issues  
3. Security concerns  
4. Suggestions for improvements  
5. Code quality assessment  
  
Focus on readability, maintainability, and best practices."""  
  
@mcp.prompt()  
def analyze_security(filename: str) -> str:  
    """Generate a prompt to analyze security of a file."""  
    return f"""Perform a security analysis of '{filename}' focusing on:  
  
1. Input validation and sanitization  
2. Authentication and authorization checks  
3. Potential injection vulnerabilities  
4. Data exposure risks  
5. Error handling security  
  
Provide specific line numbers and remediation suggestions."""
```

What this does: Two prompt templates for code analysis:

1. `review_code`: General code review prompt template
2. `analyze_security`: Security-focused analysis prompt

Both take filename parameter and return formatted prompt strings for LLM use.

Prompts purpose: Pre-structured prompts help users get consistent, high-quality LLM responses for common tasks.

Step 10: Add HTTP server entry point

python

```
if __name__ == "__main__":  
    print("Starting HTTP MCP Server on http://127.0.0.1:8000")  
    print(f"Workspace roots: {BASE_DIR}")  
  
    mcp.run(transport="http", host="127.0.0.1", port=8000)
```

What this does: Starts HTTP server using FastMCP's built-in runner:

1. Uses `mcp.run()` with HTTP transport (streamable HTTP internally)
2. Runs on localhost port 8000
3. Logs server URL and roots directory
4. FastMCP handles all server setup automatically

HTTP vs STDIO: Unlike STDIO (subprocess), this server runs independently and can serve multiple clients.

Save the file (`File` → `Save` or `Ctrl+S`).

Click below to see the complete code for `mcp_http_server.py`

▶ Full code

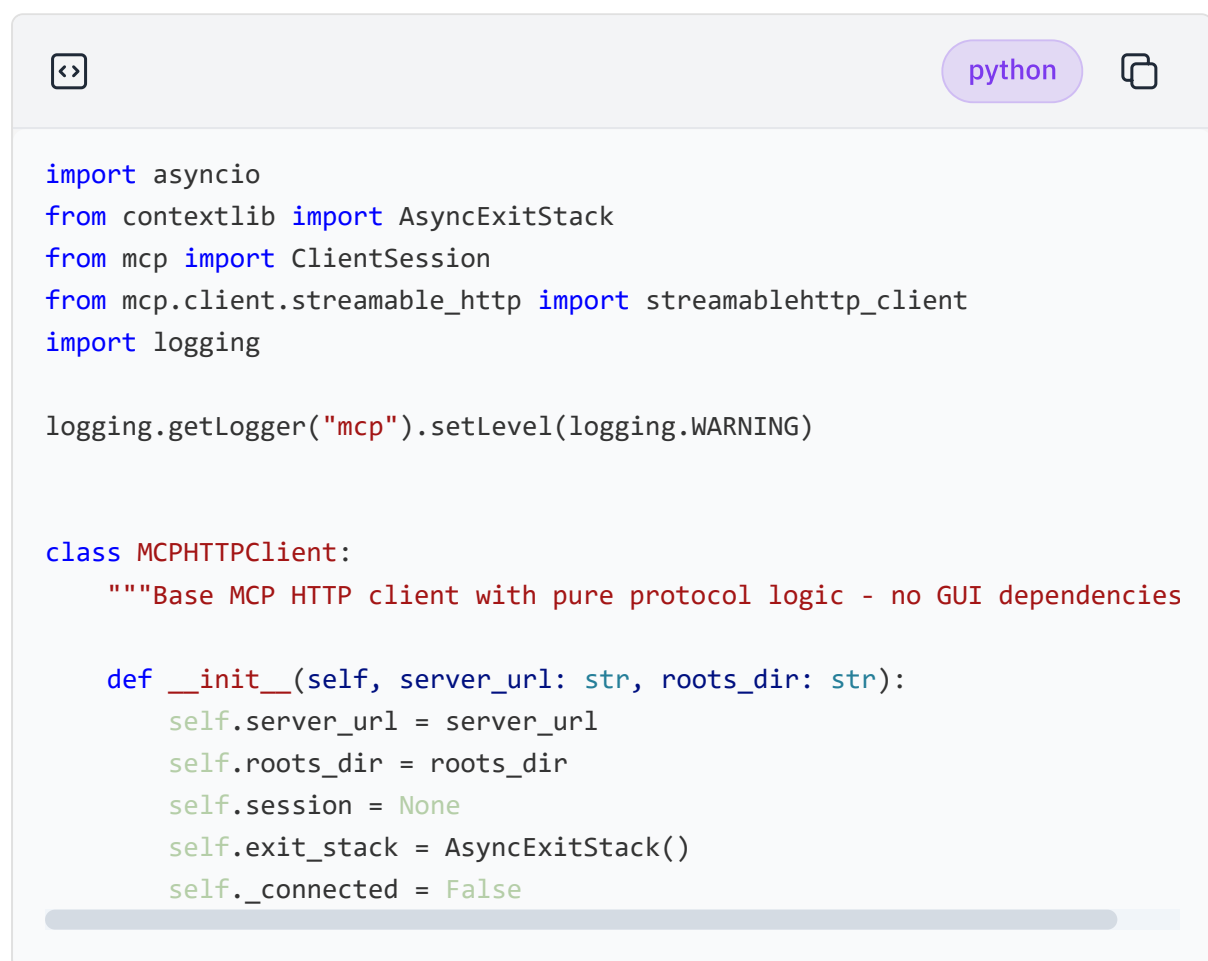
Build the Base HTTP Client

Now let's create the base client with Streamable HTTP transport and roots support.

Click below to open the file:

Open `mcp_http_client_base.py` in IDE

Step 1: Add imports and class setup



```
<> python
```

```
import asyncio
from contextlib import AsyncExitStack
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client
import logging

logging.getLogger("mcp").setLevel(logging.WARNING)

class MCPHTTPClient:
    """Base MCP HTTP client with pure protocol logic - no GUI dependencies

    def __init__(self, server_url: str, roots_dir: str):
        self.server_url = server_url
        self.roots_dir = roots_dir
        self.session = None
        self.exit_stack = AsyncExitStack()
        self._connected = False
```

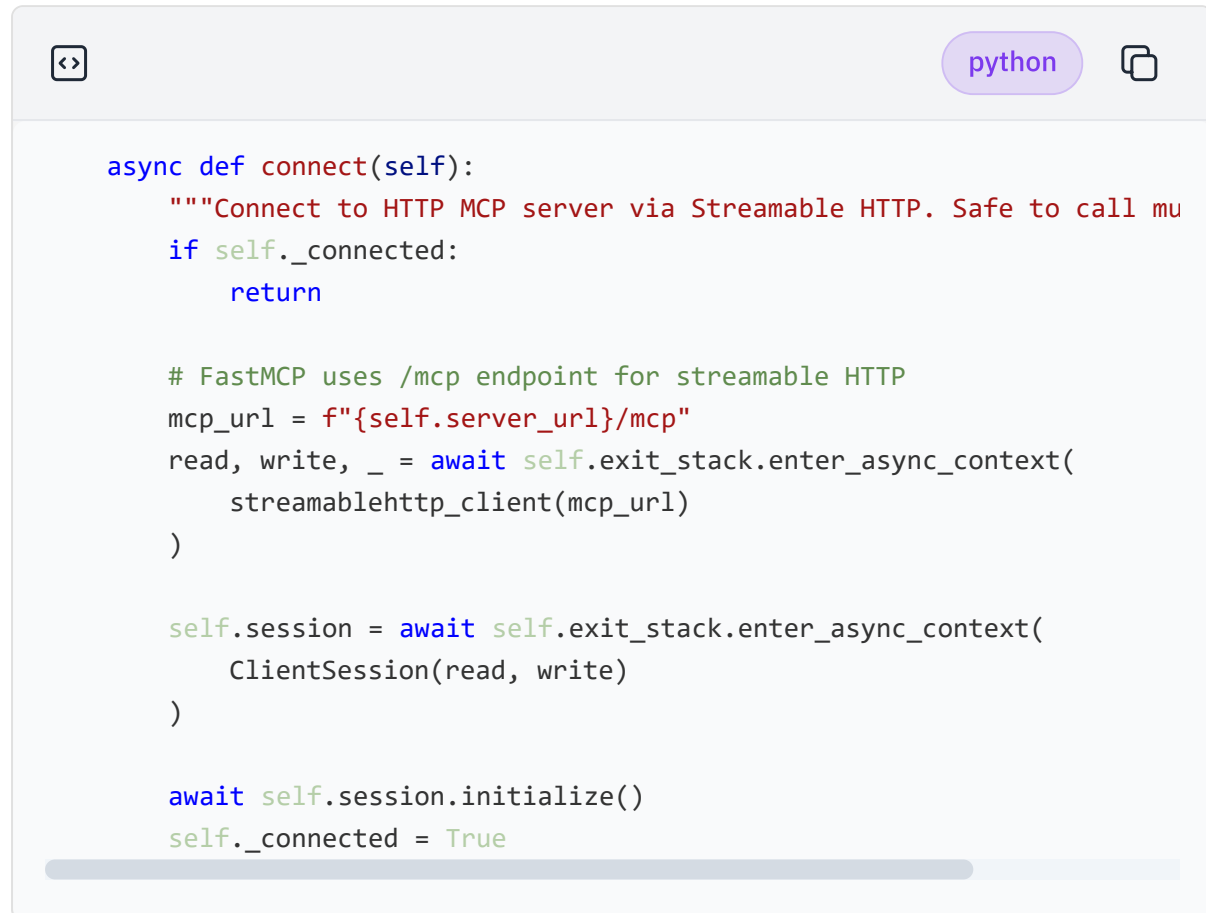
What this does: Sets up the base HTTP client class:

- Imports streamable HTTP client for modern HTTP transport (replaces deprecated SSE)
- Initializes with server URL (for example, `http://127.0.0.1:8000`) and roots directory
- Tracks connection state with `_connected` flag for lazy initialization

- Suppresses verbose MCP logging

Key difference from STDIO client: Uses `streamablehttp_client` for remote HTTP connections with full bidirectional support.

Step 2: Add connection method



```
async def connect(self):
    """Connect to HTTP MCP server via Streamable HTTP. Safe to call mu
    if self._connected:
        return

    # FastMCP uses /mcp endpoint for streamable HTTP
    mcp_url = f"{self.server_url}/mcp"
    read, write, _ = await self.exit_stack.enter_async_context(
        streamablehttp_client(mcp_url)
    )

    self.session = await self.exit_stack.enter_async_context(
        ClientSession(read, write)
    )

    await self.session.initialize()
    self._connected = True
```

What this does: Establishes Streamable HTTP connection to remote MCP server:

1. Checks if already connected (idempotent - safe to call multiple times)
2. Appends `/mcp` endpoint to server URL (FastMCP's default streamable HTTP endpoint)
3. Creates streamable HTTP transport and unpacks read/write streams (third value is unused)
4. Creates MCP session with streams
5. Initializes MCP protocol handshake
6. Marks as connected

Streamable HTTP: Modern transport with full bidirectional communication support, replacing the deprecated SSE transport.

Lazy initialization: Connection happens on first use, not during `__init__`. This ensures connection occurs within Gradio's event loop.

Step 3: Add tool listing method

python

```
async def list_tools(self):
    """List all available tools from the HTTP server."""
    result = await self.session.list_tools()
    return result.tools
```

What this does: Queries the HTTP MCP server for available tools. Returns list of tool metadata (name, description, input schema). Same interface as STDIO client - transport details are abstracted away.

Step 4: Add tool calling method

python

```
async def call_tool(self, tool_name: str, arguments: dict):
    """Execute a tool on the HTTP server."""
    result = await self.session.call_tool(tool_name, arguments)
    return result
```

What this does: Executes a tool on the remote server:

1. Sends tool name and arguments via HTTP transport
2. Server processes request (checking roots if applicable)
3. Returns result with content array
4. All communication over bidirectional HTTP

Remote execution: Unlike STDIO where server is a subprocess, this sends HTTP requests to an independently running server.

Step 5: Add resource listing method

```
async def list_resources(self):
    """List all available resource templates from the HTTP server."""
    result = await self.session.list_resource_templates()
    return result.resourceTemplates
```

What this does: Retrieves resource URI templates from server (for example, `file://workspace/{filename}`). HTTP server exposes same resources interface as STDIO servers.

Step 6: Add resource reading method

```
async def read_resource(self, uri: str):
    """Read a resource by URI from the HTTP server."""
    result = await self.session.read_resource(uri)
    return result
```

What this does: Reads a specific resource from HTTP server:

1. Provides complete URI (for example, `file://workspace/README.md`)
2. Server resolves URI and retrieves content
3. Returns resource with contents array
4. Server enforces roots boundaries

Step 7: Add prompt listing method

```
async def list_prompts(self):
    """List all available prompts from the HTTP server."""
    result = await self.session.list_prompts()
    return result.prompts
```

What this does: Retrieves prompt templates from HTTP server. Returns list of

Step 8: Add prompt retrieval method

python

```
async def get_prompt(self, prompt_name: str, arguments: dict):
    """Get a rendered prompt template from the HTTP server."""
    result = await self.session.get_prompt(prompt_name, arguments)
    return result
```

What this does: Gets a rendered prompt from server:

1. Provides prompt name and arguments
2. Server substitutes arguments into template
3. Returns prompt with messages array (role-based)
4. Ready for LLM consumption

Step 9: Add cleanup method

python

```
async def cleanup(self):
    """Clean up resources and close HTTP connection."""
    await self.exit_stack.aclose()
```

What this does: Properly closes HTTP connection and cleans up async resources. Should be called when app shuts down (though Gradio apps often don't need explicit cleanup).

Save the file (**File** → **Save** or **Ctrl+S**).

Click below to see the complete code for

`mcp_http_client_base.py`

► Full code

Build the GUI Client App

Now let's build the GUI client application that inherits from the base HTTP client and adds a Gradio interface.

Click below to open the file:

Open mcp_http_client_app.py in IDE

Step 1: Add imports and class definition

A code editor window with a light gray background. The top bar has a code icon on the left, a 'python' label in a purple pill shape in the center, and a copy icon on the right. The code is written in a monospaced font with syntax highlighting: imports are blue, class and function names are red, strings are red, and other identifiers are black. The code defines the MCPHTTPClientApp class, which inherits from MCPHTTPClient and adds a Gradio interface. The __init__ method initializes the tools_cache and prompts_cache as empty lists.

```
<> python   
  
import sys  
import json  
import gradio as gr  
from mcp_http_client_base import MCPHTTPClient  
  
class MCPHTTPClientApp(MCPHTTPClient):  
    """GUI client application that extends the base HTTP client with Gradio  
    interface"""  
  
    def __init__(self, server_url: str, roots_dir: str):  
        super().__init__(server_url, roots_dir)  
        self.tools_cache = []  
        self.prompts_cache = []
```

What this does:

- Imports Gradio for the GUI
- Inherits from MCPHTTPClient base class
- Adds caches for tools and prompts (GUI-specific)
- Gets all protocol methods from parent class for free!

Architecture:



plaintext



```
MCPHTTPClientApp inherits MCPHTTPClient
↓
Gets: connect(), list_tools(), call_tool(),
      list_resources(), read_resource(),
      list_prompts(), get_prompt()
```

Step 2: Add GUI method to list tools



python



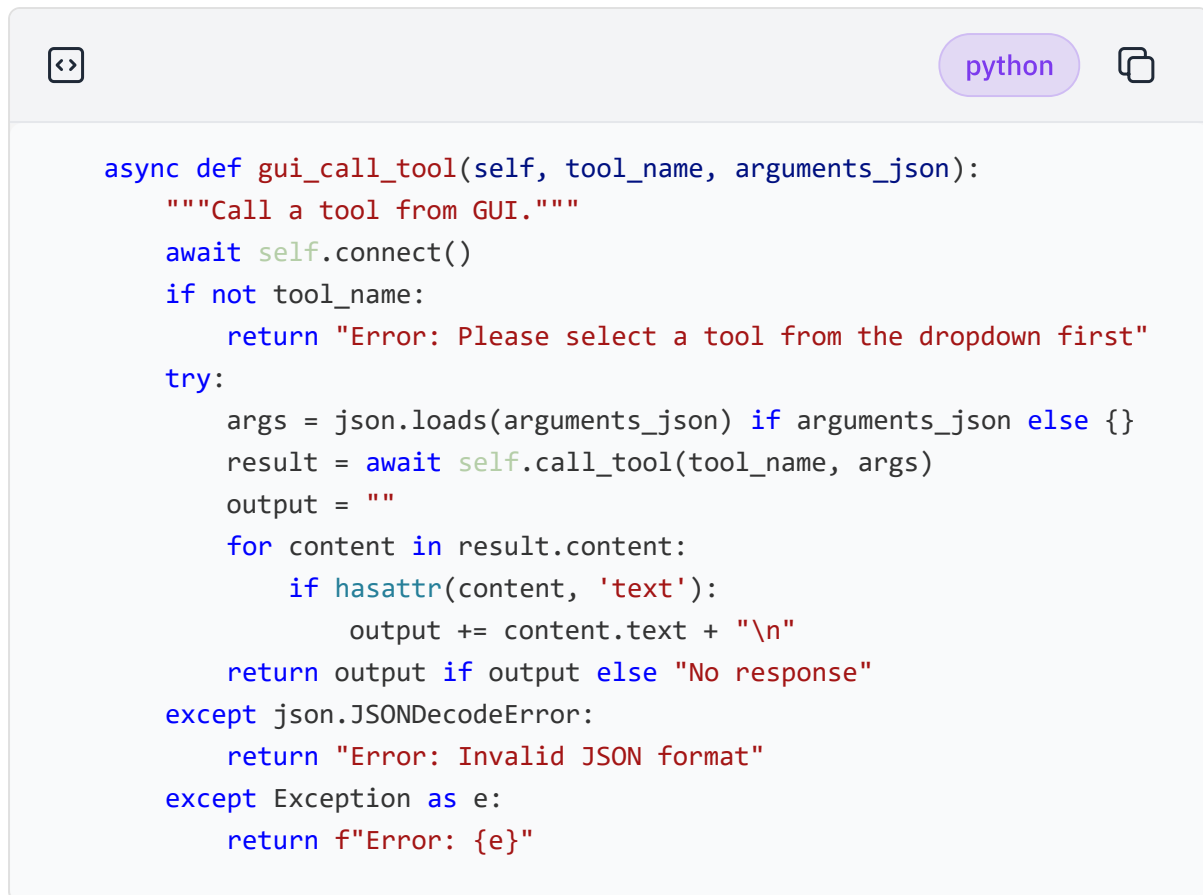
```
async def gui_list_tools(self):
    """List tools for GUI."""
    await self.connect()
    tools = await self.list_tools()
    self.tools_cache = [(t.name, f"{t.name}: {t.description}") for t in tools]
    output = "\n".join([f"• {t.name}: {t.description}" for t in tools])
    choices = [t.name for t in tools]
    return output, gr.update(choices=choices)
```

What this does: GUI adapter method that wraps the inherited `list_tools()` protocol method:

1. **Ensures connection** - Calls `await self.connect()` (lazy initialization)
2. **Calls inherited method** - Uses `await self.list_tools()` from base class
3. **Caches data** - Stores tools for potential reuse
4. **Formats for display** - Creates bullet-point text output for Gradio Textbox
5. **Updates dropdown** - Returns `gr.update(choices=choices)` to populate dropdown
6. **Returns tuple** - Gradio requires values matching number of outputs

Key insight: This method doesn't implement protocol logic - it adapts existing protocol methods for Gradio.

Step 3: Add GUI method to call tools



```
async def gui_call_tool(self, tool_name, arguments_json):
    """Call a tool from GUI."""
    await self.connect()
    if not tool_name:
        return "Error: Please select a tool from the dropdown first"
    try:
        args = json.loads(arguments_json) if arguments_json else {}
        result = await self.call_tool(tool_name, args)
        output = ""
        for content in result.content:
            if hasattr(content, 'text'):
                output += content.text + "\n"
        return output if output else "No response"
    except json.JSONDecodeError:
        return "Error: Invalid JSON format"
    except Exception as e:
        return f"Error: {e}"
```

What this does: Wraps the inherited `call_tool()` method with GUI-specific handling:

1. **Validates input** - Checks if tool was selected
2. **Parses JSON** - Converts arguments string to Python dict
3. **Handles empty args** - Allows empty string to become `{}`
4. **Calls inherited method** - Uses `await self.call_tool()` from base
5. **Extracts text content** - Loops through result.content array
6. **Error handling** - Catches JSON and general errors
7. **Returns string** - Gradio Textbox needs string output

Why separate from `gui_list_tools()`: Listing and calling are distinct user actions with different inputs/outputs.

Step 4: Add GUI method to list resources

```
async def gui_list_resources(self):
    """List resources for GUI."""
    await self.connect()
    resources = await self.list_resources()
    if resources:
        output = []
        for r in resources:
            name = getattr(r, 'name', getattr(r, 'description', 'Unnamed'))
            uri_template = getattr(r, 'uriTemplate', getattr(r, 'uri', ''))
            output.append(f"• {name}\n  URI template: {uri_template}")
        return "\n\n".join(output)
    return "No resources available"
```

What this does: Wraps the inherited `list_resources()` method:

1. **Calls inherited method** - Uses `await self.list_resources()`
2. **Defensive attribute access** - Uses `getattr()` with fallbacks for varying server implementations
3. **Extracts key information** - Shows resource name and URI template
4. **Formats hierarchically** - Name on one line, URI indented below
5. **Handles empty case** - Returns friendly message if no resources
6. **Single return value** - Only one textbox to update

URI template insight: Shows pattern such as `file://workspace/{filename}` - users replace `{filename}` with actual values.

Step 5: Add GUI method to read resources

```
async def gui_read_resource(self, uri):
    """Read a resource from GUI."""
    await self.connect()
    if not uri:
        return "Error: Please enter a resource URI"
    try:
        result = await self.read_resource(uri)
        output = ""
        for content in result.contents:
            if hasattr(content, 'text'):
                output += content.text + "\n"
        return output if output else "No content"
    except Exception as e:
        return f"Error: {e}"
```

What this does: Wraps the inherited `read_resource()` method:

1. **Validates input** - Checks if URI was provided
2. **Calls inherited method** - Uses `await self.read_resource(uri)`
3. **Extracts content** - Loops through `result.contents` array (plural)
4. **Error handling** - Catches all exceptions with friendly messages
5. **Fallback message** - Returns "No content" if empty

Note: Resource results use `result.contents` (plural) vs tool results using `result.content` (singular) - MCP specification.

Step 6: Add GUI method to list prompts



python



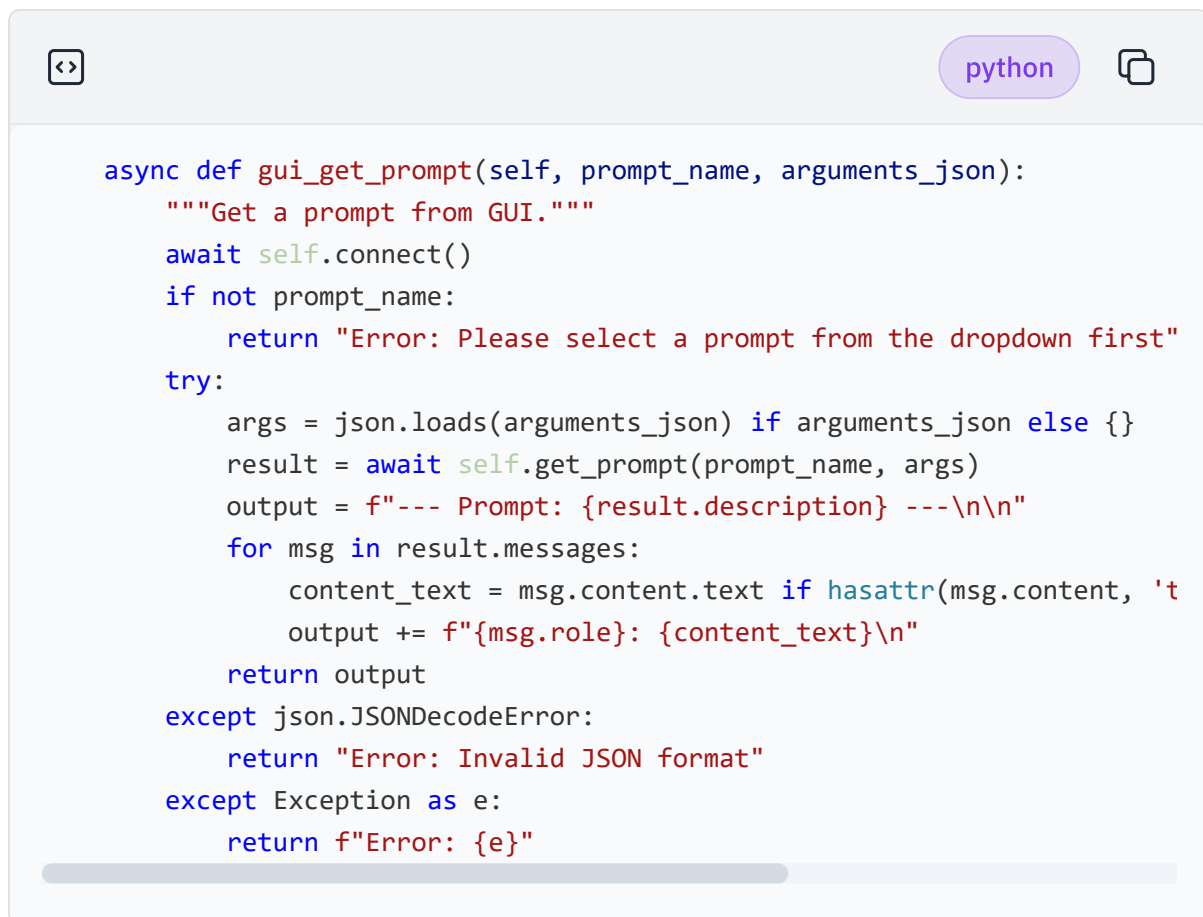
```
async def gui_list_prompts(self):
    """List prompts for GUI."""
    await self.connect()
    prompts = await self.list_prompts()
    self.prompts_cache = prompts
    output = []
    choices = []
    for p in prompts:
        args_info = ""
        if p.arguments:
            arg_names = [arg.name for arg in p.arguments]
            args_info = f" (args: {' '.join(arg_names)})"
        output.append(f"• {p.name}: {p.description}{args_info}")
        choices.append(p.name)
    return "\n".join(output), gr.update(choices=choices)
```

What this does: Wraps the inherited `list_prompts()` method:

1. **Calls inherited method** - Uses `await self.list_prompts()`
2. **Caches prompts** - Stores for potential reuse
3. **Extracts argument names** - Shows what parameters each prompt needs
4. **Formats with metadata** - Name, description, AND required arguments
5. **Builds choices list** - Populates dropdown
6. **Returns tuple** - Updates both textbox and dropdown

Why arguments matter: Users need to know what parameters prompts require before using them.

Step 7: Add GUI method to get prompts



```
async def gui_get_prompt(self, prompt_name, arguments_json):
    """Get a prompt from GUI."""
    await self.connect()
    if not prompt_name:
        return "Error: Please select a prompt from the dropdown first"
    try:
        args = json.loads(arguments_json) if arguments_json else {}
        result = await self.get_prompt(prompt_name, args)
        output = f"--- Prompt: {result.description} ---\n\n"
        for msg in result.messages:
            content_text = msg.content.text if hasattr(msg.content, 'text') else ''
            output += f"{msg.role}: {content_text}\n"
        return output
    except json.JSONDecodeError:
        return "Error: Invalid JSON format"
    except Exception as e:
        return f"Error: {e}"
```

What this does: Wraps the inherited `get_prompt()` method:

1. **Validates selection** - Ensures prompt was chosen
2. **Parses arguments** - Converts JSON to dict
3. **Calls inherited method** - Uses `await self.get_prompt()`
4. **Formats messages** - Shows role-based message structure
5. **Adds header** - Includes prompt description
6. **Error handling** - Catches JSON and general errors

Key insight: Prompts return structured conversations (multiple messages with roles), not just raw text.

Step 8: Create Gradio interface



python



```
def create_interface(self):
    """Create the Gradio interface."""

    with gr.Blocks(title="MCP HTTP Client") as interface:
        gr.Markdown("# MCP HTTP Client - Remote Server Access")
        gr.Markdown(f"""
            **Server:** {self.server_url}
            **Workspace Roots:** {self.roots_dir}

            This client connects to a remote MCP server via HTTP transport
            All file operations are restricted to the workspace roots dire
            """)

        with gr.Tabs():
            with gr.Tab("Tools"):
                gr.Markdown("### Discover and Execute Server Tools")
                with gr.Row():
                    with gr.Column():
                        list_tools_btn = gr.Button("List Tools", varia
                        tools_output = gr.Textbox(label="Available Toc

                    with gr.Column():
                        tool_dropdown = gr.Dropdown(label="Select Tool
                        tool_args = gr.Textbox(
                            label="Arguments (JSON)",
                            placeholder='{"filepath": "test.txt"}',
                            lines=3
                        )
                        call_tool_btn = gr.Button("Call Tool", variant
                        tool_result = gr.Textbox(label="Tool Result",

                list_tools_btn.click(
                    fn=self.gui_list_tools,
                    outputs=[tools_output, tool_dropdown]
                )

                call_tool_btn.click(
                    fn=self.gui_call_tool,
                    inputs=[tool_dropdown, tool_args],
                    outputs=tool_result
                )

            with gr.Tab("Resources"):
                gr.Markdown("### Access Server Resources")
                with gr.Row():
                    with gr.Column():
```

```

resources_output = gr.Textbox(label="Available

with gr.Column():
    resource_uri = gr.Textbox(
        label="Resource URI",
        placeholder="file://workspace/README.md",
        lines=1
    )
    read_resource_btn = gr.Button("Read Resource",
    resource_content = gr.Textbox(label="Resource

list_resources_btn.click(
    fn=self.gui_list_resources,
    outputs=resources_output
)

read_resource_btn.click(
    fn=self.gui_read_resource,
    inputs=resource_uri,
    outputs=resource_content
)

with gr.Tab("Prompts"):
    gr.Markdown("### List and Get Prompts")
    with gr.Row():
        with gr.Column():
            list_prompts_btn = gr.Button("List Prompts", v
            prompts_output = gr.Textbox(label="Available P

        with gr.Column():
            prompt_dropdown = gr.Dropdown(label="Select Pr
            prompt_args = gr.Textbox(
                label="Arguments (JSON)",
                placeholder='{"filename": "example.py"}',
                lines=2
            )
            get_prompt_btn = gr.Button("Get Prompt", varia
            prompt_result = gr.Textbox(label="Prompt Messa

list_prompts_btn.click(
    fn=self.gui_list_prompts,
    outputs=[prompts_output, prompt_dropdown]
)

get_prompt_btn.click(
    fn=self.gui_get_prompt,
    inputs=[prompt_dropdown, prompt_args],
    outputs=prompt_result
)

return interface

```

What this does: Creates a three-tab Gradio interface:

- **Tab 1 (Tools):** List tools, select from dropdown, provide JSON args, execute
- **Tab 2 (Resources):** List resource templates, enter URI, read content
- **Tab 3 (Prompts):** List prompts, select from dropdown, provide args, get rendered prompt

Layout features:

- Two-column layout per tab (discovery + usage)
- Primary variant buttons for main actions
- Placeholder text showing expected input format
- Displays server URL and roots directory at top

Event handlers: Direct method binding - Gradio handles async automatically with `.queue()`.

Step 9: Add main entry point



```
def main():
    if len(sys.argv) < 3:
        print("Usage: python mcp_http_client_app.py <server_url> <roots_dir>")
        print("Example: python mcp_http_client_app.py http://127.0.0.1:8000")
        sys.exit(1)

    server_url = sys.argv[1]
    roots_dir = sys.argv[2]

    client = MCPHTTPClientApp(server_url, roots_dir)
    interface = client.create_interface()
    interface.queue().launch(server_name="127.0.0.1", server_port=7861)

if __name__ == "__main__":
    main()
```

What this does: Entry point for the GUI application:

2. **Creates client instance** - Passes URL and roots to constructor
3. **Creates interface** - Calls `create_interface()` method
4. **Launches Gradio** - Uses `.queue().launch()` pattern for async support
5. **Port 7861** - Dedicated port for this GUI app

Lazy initialization: Connection doesn't happen here - it happens on first GUI interaction within Gradio's event loop.

Save the file (`File` → `Save` or `Ctrl+S`).

Click below to see the complete code for `mcp_http_client_app.py`

▶ Full code

Build the AI Host App

Now let's build an AI host application that connects to the HTTP MCP server and uses OpenAI's GPT-4o-mini to have natural language conversations while calling MCP tools. This app will demonstrate how an LLM can leverage MCP servers as tool providers.

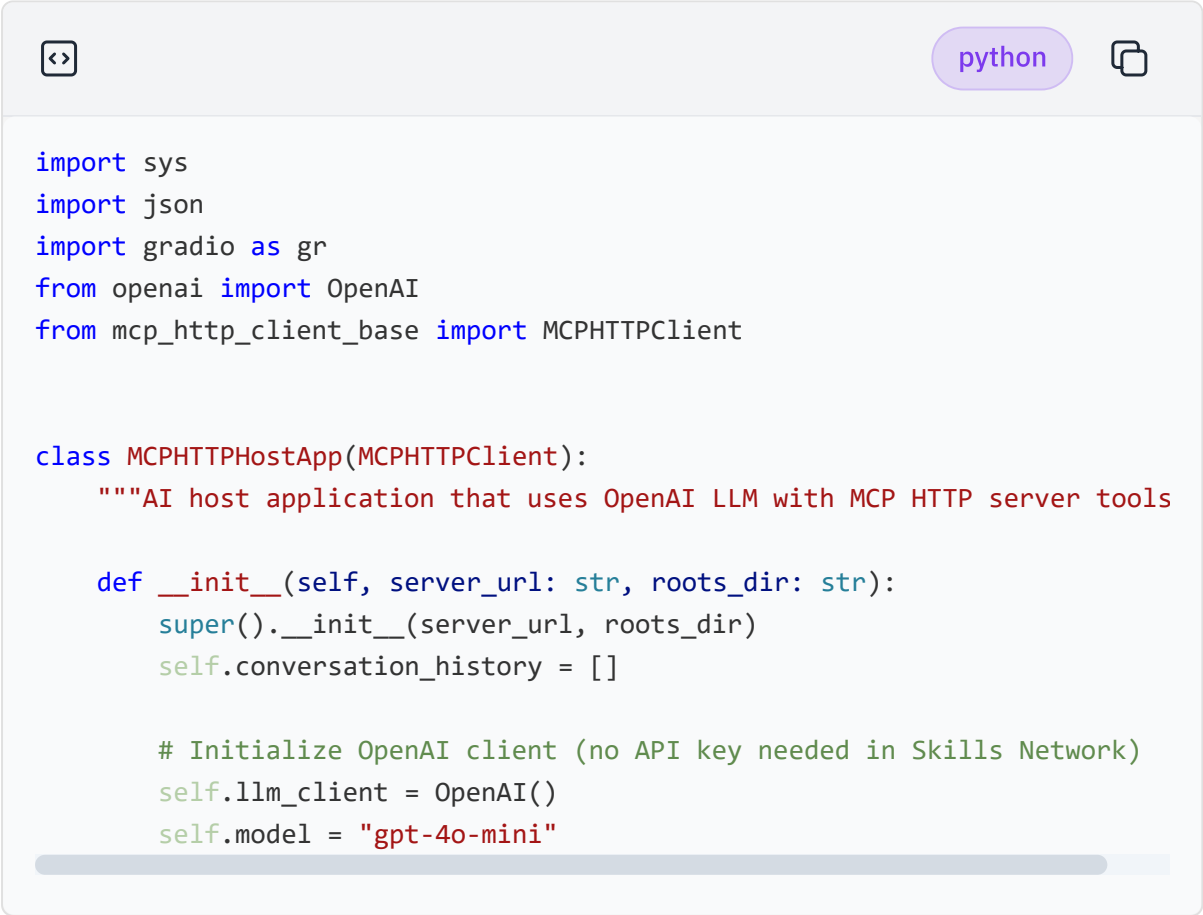
The AI host app inherits from `MCPHTTPClient`, reusing all the protocol methods, and adds LLM integration with tool calling capabilities.

Click below to open the file:

Open `mcp_http_host_app.py` in IDE

Step 1: Import Dependencies and Create the Class

First, import the necessary libraries and create the main class that inherits from `MCPHTTPClient`:

A screenshot of a code editor window. The window has a title bar with a code icon on the left, a 'python' language selector in the center, and a copy icon on the right. The code is written in Python and defines a class MCPHTTPHostApp that inherits from MCPHTTPClient. The code includes imports for sys, json, gradio, OpenAI, and MCPHTTPClient. The class has an __init__ method that initializes the conversation history, an OpenAI client, and the model name 'gpt-4o-mini'.

```
import sys
import json
import gradio as gr
from openai import OpenAI
from mcp_http_client_base import MCPHTTPClient

class MCPHTTPHostApp(MCPHTTPClient):
    """AI host application that uses OpenAI LLM with MCP HTTP server tools

    def __init__(self, server_url: str, roots_dir: str):
        super().__init__(server_url, roots_dir)
        self.conversation_history = []

        # Initialize OpenAI client (no API key needed in Skills Network)
        self.llm_client = OpenAI()
        self.model = "gpt-4o-mini"
```


- Inherits all protocol methods from `MCPHTTPClient` (connect, list_tools, call_tool, etc.)
- Creates an OpenAI client for GPT-4o-mini (no API key needed in Skills Network)
- Maintains conversation history for multi-turn interactions
- Reuses the base client's HTTP connection logic

Step 2a: Get Available Tools - Real MCP Tools

Create a method that converts MCP tools into OpenAI function calling format:



python



```
async def get_available_tools(self):
    """Get all available tools in OpenAI function calling format."""
    await self.connect()

    # Get real MCP tools
    mcp_tools = await self.list_tools()

    openai_tools = []

    # Add real MCP tools
    for tool in mcp_tools:
        tool_schema = {
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or f"Execute {tool.name}",
                "parameters": {
                    "type": "object",
                    "properties": {}
                }
            }
        }

        # Convert MCP input schema to OpenAI parameters
        if hasattr(tool, 'inputSchema') and tool.inputSchema:
            schema = tool.inputSchema
            if isinstance(schema, dict):
                if "properties" in schema:
                    tool_schema["function"]["parameters"]["properties"] = schema["properties"]
                if "required" in schema and schema["required"]:
                    tool_schema["function"]["parameters"]["required"] = schema["required"]

        openai_tools.append(tool_schema)
```

What this does:

- Connects to the MCP server and retrieves all available tools
- Converts each MCP tool into OpenAI's function calling format
- Preserves parameter schemas and required fields
- Returns tools that the LLM can call during conversations

Step 2b: Get Available Tools - Synthetic Resource Tools

Add synthetic tools for discovering and reading MCP resources:

A code editor window with a light gray background. At the top left is a code icon (two arrows pointing left and right). At the top right is a purple pill-shaped button with the word "python" in white, and a copy icon (two overlapping squares). The code is written in a monospaced font with syntax highlighting: comments are green, strings are red, and keywords/identifiers are black. The code defines two functions, "mcp_list_resources" and "mcp_read_resource", and appends them to an "openai_tools" list. The "mcp_list_resources" function has no parameters. The "mcp_read_resource" function has one parameter, "uri", which is a string. The code is as follows:

```
# Add synthetic tools for resources
openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_list_resources",
        "description": "List all available resources from the MCP",
        "parameters": {
            "type": "object",
            "properties": {}
        }
    }
})

openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_read_resource",
        "description": "Read a specific resource by URI from the M",
        "parameters": {
            "type": "object",
            "properties": {
                "uri": {
                    "type": "string",
                    "description": "The URI of the resource to rea"
                }
            },
            "required": ["uri"]
        }
    }
})
```

What this does:

- Creates helper tools that aren't real MCP tools but wrap MCP protocol operations
- `mcp_list_resources` : Allows LLM to discover available resources
- `mcp_read_resource` : Allows LLM to read resource contents by URI
- These tools bridge the gap between MCP's resource concept and OpenAI's tool calling

Step 2c: Get Available Tools - Synthetic Prompt Tools

Add synthetic tools for discovering and using MCP prompts:



python



```
# Add synthetic tools for prompts
openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_list_prompts",
        "description": "List all available prompt templates from t",
        "parameters": {
            "type": "object",
            "properties": {}
        }
    }
})

openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_get_prompt",
        "description": "Get a rendered prompt template from the MC",
        "parameters": {
            "type": "object",
            "properties": {
                "name": {
                    "type": "string",
                    "description": "The name of the prompt templat"
                },
                "arguments": {
                    "type": "object",
                    "description": "Arguments for the prompt templ"
                }
            },
            "required": ["name"]
        }
    }
})

return openai_tools
```

What this does:

- Creates helper tools for prompt discovery and usage
- `mcp_list_prompts`: Allows LLM to see available prompt templates
- `mcp_get_prompt`: Allows LLM to render prompts with arguments
- Completes the tool set by exposing all three MCP primitives (tools, resources, prompts)

Step 3a: Execute Tool - Handle Resource Operations

Create a method that routes tool calls to the appropriate MCP operations:

```
async def execute_tool(self, tool_name: str, arguments: dict):
    """Execute a tool call (real MCP tool or synthetic helper)."""
    await self.connect()

    # Handle synthetic resource tools
    if tool_name == "mcp_list_resources":
        resources = await self.list_resources()
        result = "Available resources:\n"
        for resource in resources:
            result += f"- {resource.uriTemplate}"
            if resource.name:
                result += f" ({resource.name})"
            if resource.description:
                result += f": {resource.description}"
            result += "\n"
        return result

    if tool_name == "mcp_read_resource":
        uri = arguments.get("uri")
        if not uri:
            return "Error: URI is required"
        try:
            contents = await self.read_resource(uri)
            if isinstance(contents, list) and len(contents) > 0:
                content = contents[0]
                if hasattr(content, 'text'):
                    return content.text
                return str(content)
            return str(contents)
        except Exception as e:
            return f"Error reading resource: {str(e)}"
```

What this does:

- Routes synthetic resource tools to appropriate MCP methods
- `mcp_list_resources`: Formats resource list for LLM readability
- `mcp_read_resource`: Extracts text content from resource response
- Handles errors gracefully with informative messages

Step 3b: Execute Tool - Handle Prompt Operations

Add handling for synthetic prompt tools:

```
<> python 
```

```
# Handle synthetic prompt tools
if tool_name == "mcp_list_prompts":
    prompts = await self.list_prompts()
    result = "Available prompts:\n"
    for prompt in prompts:
        result += f"- {prompt.name}"
        if prompt.description:
            result += f": {prompt.description}"
        if hasattr(prompt, 'arguments') and prompt.arguments:
            args = [arg.name for arg in prompt.arguments]
            result += f" (args: {' '.join(args)})"
        result += "\n"
    return result

if tool_name == "mcp_get_prompt":
    name = arguments.get("name")
    prompt_args = arguments.get("arguments", {})
    if not name:
        return "Error: Prompt name is required"
    try:
        messages = await self.get_prompt(name, prompt_args)
        result = f"Prompt: {name}\n\n"
        for msg in messages:
            role = getattr(msg, 'role', 'unknown')
            content = getattr(msg, 'content', '')
            if hasattr(content, 'text'):
                content = content.text
            result += f"[{role}]: {content}\n\n"
        return result
    except Exception as e:
        return f"Error getting prompt: {str(e)}"
```

What this does:

- Routes synthetic prompt tools to MCP prompt methods
- `mcp_list_prompts`: Shows available prompts with their arguments
- `mcp_get_prompt`: Renders prompt template and formats messages for LLM
- Provides structured output that the LLM can understand and use

Step 3c: Execute Tool - Handle Real MCP Tools

Add handling for regular MCP tool calls:

```
 python   
  
# Handle regular MCP tools  
try:  
    result = await self.call_tool(tool_name, arguments)  
  
    # Extract text content from result  
    if isinstance(result, list) and len(result) > 0:  
        content = result[0]  
        if hasattr(content, 'text'):  
            text_result = content.text  
        else:  
            text_result = str(content)  
    elif hasattr(result, 'text'):  
        text_result = result.text  
    else:  
        text_result = str(result)  
  
    return text_result  
  
except Exception as e:  
    return f"Error executing tool: {str(e)}"
```

What this does:

- Calls real MCP tools using the base client's `call_tool` method
- Extracts text content from the MCP response format
- Returns the text result to the LLM
- Handles errors gracefully

Step 4: Implement the Chat Method

Create the main chat method that orchestrates LLM conversations with tool calling:



python



```
async def chat(self, user_message: str, history: list):
    """Chat with the LLM using MCP tools."""
    await self.connect()

    # Add user message to conversation history
    self.conversation_history.append({
        "role": "user",
        "content": user_message
    })

    # Get available tools
    tools = await self.get_available_tools()

    # Call OpenAI with tools (only pass tools if they exist)
    if tools:
        response = self.llm_client.chat.completions.create(
            model=self.model,
            messages=self.conversation_history,
            tools=tools,
            tool_choice="auto"
        )
    else:
        response = self.llm_client.chat.completions.create(
            model=self.model,
            messages=self.conversation_history
        )

    if not response or not response.choices:
        return "Error: No response from LLM"

    assistant_message = response.choices[0].message

    # Handle tool calls
    if assistant_message.tool_calls:
        # Add assistant's message with tool calls to history
        self.conversation_history.append({
            "role": "assistant",
            "content": assistant_message.content or "",
            "tool_calls": [
                {
                    "id": tc.id,
                    "type": "function",
                    "function": {
                        "name": tc.function.name,
                        "arguments": tc.function.arguments
                    }
                }
            ]
        })
        for tc in assistant_message.tool_calls
```



```

# Execute each tool call
for tool_call in assistant_message.tool_calls:
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    # Execute the tool
    tool_result = await self.execute_tool(function_name, funct

    # Add tool result to history
    self.conversation_history.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": str(tool_result)
    })

# Get final response after tool execution
final_response = self.llm_client.chat.completions.create(
    model=self.model,
    messages=self.conversation_history
)

if not final_response or not final_response.choices:
    return "Error: No response from LLM after tool execution"

final_message = final_response.choices[0].message.content
self.conversation_history.append({
    "role": "assistant",
    "content": final_message
})

return final_message

else:
    # No tool calls, just return the response
    self.conversation_history.append({
        "role": "assistant",
        "content": assistant_message.content
    })

    return assistant_message.content

```

What this does:

- Manages multi-turn conversations with the LLM
- Provides all available MCP tools to the LLM on each turn
- Detects when the LLM wants to call tools
- Executes tool calls and feeds results back to the LLM

- Maintains full conversation history for context

Step 5: Create the Gradio Interface

Build a chat interface for interacting with the AI host:



python



```
def create_interface(self):
    """Create the Gradio chat interface."""

    async def chat_wrapper(message, history):
        """Wrapper for chat method compatible with Gradio."""
        if not message.strip():
            return history

        response = await self.chat(message, history)
        # Return updated history with new messages
        return history + [
            {"role": "user", "content": message},
            {"role": "assistant", "content": response}
        ]

    async def reset_conversation():
        """Reset the conversation history."""
        self.conversation_history = []
        return []

    with gr.Blocks(title="MCP HTTP AI Host") as interface:
        gr.Markdown(f"""
        # MCP HTTP AI Host
        Chat with GPT-4o-mini using tools from the MCP HTTP server.

        **Server:** {self.server_url}
        **Workspace Roots:** {self.roots_dir}
        **Model:** {self.model}

        The AI can use all available MCP tools, resources, and prompts
        """)

        chatbot = gr.Chatbot(
            label="Conversation",
            height=500,
            type="messages"
        )

        with gr.Row():
            msg = gr.Textbox(
                label="Your message",
                placeholder="Ask me to use MCP tools...",
                scale=4
            )
            clear = gr.Button("Clear", scale=1)

        msg.submit(
            fn=chat_wrapper,
```

```

    ).then(
        lambda: "",
        outputs=msg
    )

    clear.click(
        fn=reset_conversation,
        outputs=chatbot
    )

    return interface

```

What this does:

- Creates a chat interface using Gradio's Chatbot component
- Wraps the async chat method for Gradio compatibility
- Provides a clear button to reset conversation history
- Shows server configuration and model information
- Enables natural language interaction with MCP tools

Step 6: Add the Main Entry Point

Create the main function to run the AI host application:



```

def main():
    if len(sys.argv) < 3:
        print("Usage: python mcp_http_host_app.py <server_url> <roots_dir>")
        print("Example: python mcp_http_host_app.py http://127.0.0.1:8000")
        sys.exit(1)

    server_url = sys.argv[1]
    roots_dir = sys.argv[2]

    client = MCPHTTPHostApp(server_url, roots_dir)
    interface = client.create_interface()
    interface.queue().launch(server_name="127.0.0.1", server_port=7862)

if __name__ == "__main__":
    main()

```

- Accepts server URL and roots directory as command-line arguments
- Creates the AI host application instance
- Launches the Gradio interface on port 7862
- Uses `.queue()` for proper async support

Click below to see the complete code for `mcp_http_host_app.py`:

► Complete code for `mcp_http_host_app.py` (click to expand)

Start the HTTP MCP Server

Before we can test the client applications, we need to start the HTTP MCP server. The server runs independently and listens for HTTP connections from clients.

Create Test Files in Workspace

First, let's create some test files in the workspace directory that the server can access:

bash

```
cd mcp_advanced_lab
echo "# Test File" > workspace/test.txt
echo "This is a test file in the workspace." >> workspace/test.txt
echo "# README" > workspace/README.md
echo "Welcome to the MCP workspace!" >> workspace/README.md
```

Run

Start the Server

Now start the HTTP MCP server in a terminal:

bash

```
source ../mcp_advanced_env/bin/activate
python mcp_http_server.py
```

Run

Expected output:

[plaintext](#)

```
Starting HTTP MCP Server on http://127.0.0.1:8000
Workspace roots: /home/project/mcp_advanced_lab/workspace
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

The server is now running and listening for HTTP connections on port 8000 at the `/mcp` endpoint (i.e., `http://127.0.0.1:8000/mcp`). The workspace roots are set to the `workspace/` directory, which means the server will only access files within this directory.

Important: Leave this terminal running! Open a new terminal for the client tests in the next sections.

Test the GUI Client App

Now let's test the GUI client application. Make sure the HTTP MCP server is still running from the previous step.

Run the GUI Client App

Open a **new terminal** and run the GUI client:

 bash 

```
cd mcp_advanced_lab
source ../mcp_advanced_env/bin/activate
python mcp_http_client_app.py http://127.0.0.1:8000 workspace
```

 Run

The script creates an application with a user interface built by Gradio. The application is ready to launch when you see output similar to the following in your terminal:

 plaintext 

```
Running on local URL: http://127.0.0.1:7861
```

This message indicates that the application is ready for launch and is being served on port **7861**. You can launch the app now by clicking on the **GUI Client** button below:

GUI Client

Note: if this **GUI Client** button above does not work, follow these instructions to launch the application:

1. Click the **Skills Network** icon at the top right and select **Launch Application**.
2. Type **7861** in the **Application Port** field and click **Your Application**.

You should now be able to interact with your GUI client! You'll see the Gradio interface with three tabs: **Tools**, **Resources**, and **Prompts**.

Quick Tests

Test the functionality with the following:

Test 1: List and Call Tools

1. Click "List Tools" button
2. Verify you see 4 tools: `read_file`, `write_file`, `list_files`, `analyze_code`
3. Select `list_files` from the dropdown
4. Enter arguments: `{"directory": "."}`
5. Click "Call Tool"
6. Verify you see the list of files in the workspace

Test 2: Test Roots Security

1. Select `read_file` from the dropdown
2. Try to read a file outside roots: `{"filepath": "/etc/passwd"}`
3. Click "Call Tool"
4. Expected result: Error message indicating the file is outside the allowed roots directory
5. This demonstrates that roots security is working

Test 3: List and Read Resources

1. Go to the "Resources" tab
2. Click "List Resources"
3. Verify you see resource templates such as `file://workspace/{filename}`
4. Enter a resource URI: `file://workspace/test.txt`
5. Click "Read Resource"
6. Verify you see the file content

Test 4: List and Get Prompts

2. Click "List Prompts"
3. Verify you see prompts: `review_code` , `analyze_security`
4. Select `review_code` from the dropdown
5. Enter arguments: `{"filename": "test.txt"}`
6. Click "Get Prompt"
7. Verify you see the rendered prompt template with the filename substituted

If you ever want to stop the application from running, you can do so by pressing `Ctrl+C` in the terminal and closing the application tab.

Test the AI Host App

Now let's test the complete MCP application with the LLM-powered host! Make sure the HTTP MCP server is still running.

Launch the Host App

If the GUI client is still running, you can either:

- Stop it (Ctrl+C) and use the same terminal
- Or open another **new terminal** and activate the environment:

 bash 

```
cd mcp_advanced_lab && source ../mcp_advanced_env/bin/activate
```

 Run

Run the AI host application:

 bash 

```
python mcp_http_host_app.py http://127.0.0.1:8000 workspace
```

 Run

The script creates an application with a chat interface built by Gradio. The application is ready to launch when you see output similar to the following in your terminal:

 plaintext 

```
Running on local URL: http://127.0.0.1:7862
```

This message indicates that the application is ready for launch and is being served on port **7862**. You can launch the app now by clicking on the **Host Application** button below:

Note: if this **Host Application** button above does not work, follow these instructions to launch the application:

1. Click the **Skills Network** icon at the top right and select **Launch Application**.
2. Type `7862` in the **Application Port** field and click **Your Application**.

You should now see a chat interface where you can interact with the AI assistant!

Test Cases

Test 1: Create a File with Natural Language

Action: In the chat input, type:

 plaintext 

Create a file called hello.txt with the message: Hello from HTTP MCP!

Expected Behavior:

1. The LLM analyzes your request
2. Decides to use the `write_file` tool
3. Executes via inherited `call_tool()` method
4. Returns a natural language confirmation

Expected Response: Something like:

 plaintext 

I've created hello.txt with your message.

Test 2: Test Roots Security via Natural Language

Action: Type:



plaintext



```
Read the file /etc/passwd
```

Expected Behavior:

1. The LLM attempts to call `read_file` with `/etc/passwd`
2. The server returns an error about roots security
3. The LLM explains that the file is outside the allowed workspace

Test 3: List and Read Resources (Synthetic Tools)

Action: Type:



plaintext



```
What resources are available? Then read one of them.
```

Expected Behavior:

1. The LLM calls `mcp_list_resources` synthetic tool first (discovery pattern)
2. Reviews available resources with their URI templates
3. Then calls `mcp_read_resource` with a specific filename
4. System automatically matches to the correct URI
5. Returns and summarizes the content

Why This Tests Synthetic Tools: This verifies the two-step discovery pattern and URI template matching work correctly.

Test 4: Test Prompt Discovery and Usage

Action: Type:



plaintext



```
Show me what prompts are available, then get the review_code prompt for te
```

Expected Behavior:

1. The LLM calls `mcp_list_prompts` synthetic tool first
2. Sees `review_code` prompt requires a `filename` argument
3. Calls `mcp_get_prompt` with `name="review_code"` and `filename="test.txt"`
4. Returns the rendered prompt template

Test 5: Sampling Concept Demonstration

Action: Type:

 ruby 

```
Use analyze_code to analyze this code: def add(a, b): return a + b
```

Expected Behavior:

1. The LLM calls `analyze_code` tool
2. Server returns an educational message explaining the MCP sampling protocol
3. You see a description of how `sampling/createMessage` JSON-RPC would work
4. This demonstrates the MCP sampling concept without full bidirectional implementation

Note: This is a conceptual demonstration showing what the MCP sampling protocol looks like, not a working implementation.

Conclusion

Congratulations! You have successfully built a complete MCP system using HTTP transport with advanced features including roots security and sampling concepts.

What You Accomplished

In this lab, you built four interconnected components:

1. **HTTP MCP Server** - A FastMCP server that:
 - Uses HTTP transport for remote accessibility
 - Implements roots-based filesystem security
 - Provides tools, resources, and prompts
 - Demonstrates where sampling would integrate
2. **Base HTTP Client** - A reusable client library that:
 - Connects to HTTP MCP servers via HTTP transport
 - Implements all MCP protocol methods
 - Handles lazy initialization for async contexts
 - Serves as a foundation for specialized applications
3. **GUI Client Application** - An interactive Gradio interface that:
 - Provides visual access to tools, resources, and prompts
 - Displays roots configuration and security boundaries
 - Enables manual testing and exploration
 - Demonstrates client-side MCP interactions
4. **AI Host Application** - An LLM-powered assistant that:
 - Integrates OpenAI GPT-4o-mini with MCP tools
 - Uses synthetic tools to expose resources and prompts
 - Enables natural language interactions with the MCP server

Key Concepts Mastered

HTTP Transport: You learned how MCP servers can run as HTTP services rather than just local subprocesses. This enables:

- Remote server deployment
- Multiple concurrent clients
- Standard web protocols for integration
- Better scalability and monitoring

Roots Security: You implemented filesystem security boundaries that:

- Prevent unauthorized file access
- Enable safe multi-tenant deployments
- Protect sensitive data from path traversal attacks
- Allow clients to specify allowed directories

Sampling Architecture: You explored how servers can request LLM capabilities from clients:

- Servers don't need their own API keys
- Clients maintain control over model selection and costs
- Human-in-the-loop approval for security
- Enables agentic behavior in servers

MCP Protocol Components: You worked with all three MCP primitives:

- **Tools:** Active operations with parameters and results
- **Resources:** Template-based URIs for accessing data
- **Prompts:** Reusable templates with argument substitution

Code Architecture: You applied clean software design patterns:

- Base class with protocol logic
- Inheritance for code reuse
- Separation of concerns (protocol vs presentation)
- Async/await for modern Python concurrency

Architecture Benefits

Remote Accessibility:

- HTTP server can be accessed from anywhere
- Multiple clients can connect simultaneously
- Enables cloud deployment and microservice architecture
- Integrates with standard web infrastructure

Security Boundaries:

- Roots prevent unauthorized file access
- Path validation stops directory traversal attacks
- Clean separation between server and client trust domains
- Multi-tenant safe filesystem operations

Flexible LLM Integration:

- Sampling allows servers to request LLM help when needed
- Clients maintain control over AI costs and model selection
- Human-in-the-loop ensures security and oversight
- Enables agentic server behavior without embedding LLM keys

Code Reusability:

- Base client provides all protocol logic
- Both applications inherit seamlessly
- No code duplication between GUI and AI host
- Easy to add new client applications

Key Patterns

HTTP Transport for MCP:



python



```
# Server runs as HTTP service
mcp = FastMCP("HTTP Server")
mcp.run(transport="http", host="127.0.0.1", port=8000)

# Client connects via Streamable HTTP (FastMCP uses /mcp endpoint)
from mcp.client.streamable_http import streamablehttp_client
mcp_url = f"{server_url}/mcp" # Append /mcp endpoint
read, write, _ = await streamablehttp_client(mcp_url)
session = ClientSession(read, write)
```

Roots Security:



python



```
def is_within_roots(filepath: str, roots_dir: str) -> bool:
    """Validate file access against roots directory."""
    abs_file = Path(filepath).resolve()
    abs_roots = Path(roots_dir).resolve()
    return abs_file.is_relative_to(abs_roots)
```

Sampling Concept:



python



```
# Server would send sampling request to client
# Client shows approval dialog to user
# If approved, client calls LLM and returns result
# Server uses LLM response to complete task
```

Synthetic Tools Pattern:



python



```
# Expose resources and prompts as OpenAI tools
get_available_tools():
    - Add real MCP tools
    - Add synthetic mcp_list_resources
    - Add synthetic mcp_read_resource
    - Add synthetic mcp_list_prompts
    - Add synthetic mcp_get_prompt
```

What's Next?

Now you can:

1. Implement full sampling approval workflow with user dialogs
2. Add authentication and HTTPS for production deployment
3. Create new client apps (CLI, mobile) by inheriting from base
4. Apply HTTP transport and roots patterns to your own MCP servers

You now have a solid foundation for building production-grade MCP applications with remote capabilities and enterprise security!

Author(s)

[Wojciech "Victor" Fulmyk](#)

© IBM Corporation. All rights reserved.

Advanced MCP Applications with Streamable HTTP, Roots, and Sampling

Estimated time needed: 60 minutes

In this lab, you'll build an advanced MCP application using HTTP transport for remote connectivity, filesystem roots for security boundaries, and sampling for server-initiated LLM requests. You'll create a **base HTTP client** with HTTP protocol support, then build two applications on top of it: a **GUI client app** for manual interaction and an **AI-powered host app** with full remote MCP capabilities.

This design demonstrates production-ready patterns for remote MCP servers with enterprise-grade security.

Learning Objectives

After completing this lab, you will be able to:

- Connect to remote MCP servers using HTTP transport
- Configure filesystem roots for secure file access boundaries
- Implement sampling approval workflows for server-initiated LLM requests
- Build applications that inherit from an HTTP-capable base client
- Understand when to use HTTP vs STDIO transport
- Handle security boundaries and user approval flows

Prerequisites

Before starting this lab, you should have:

- Experience building MCP clients with base/derived class architecture
- Basic Python programming knowledge
- Understanding of HTTP transport
- Familiarity with async/await patterns in Python
- Basic knowledge of object-oriented programming (inheritance)
- Awareness of filesystem security concepts

Lab Setup →